UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

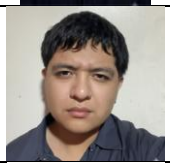
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

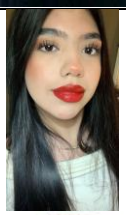
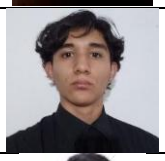
SISTEMAS OPERATIVOS

Actividad Fundamental 3: Almacenaje, memoria y archivos

DOCENTE: DRA. NORMA EDITH MARIN MARTINEZ

EQUIPO N°3

Integrantes	Foto	Matricula	% de participación
Camacho Moreno Jesús Emmanuel		2153842	100%
Cárdenas Vázquez Carlos Tomas		2226997	100%
Cerda Herrera Juan Kristof		2050864	100%
Espindola Martínez Juan		2153695	100%
Garza Arizpe Emilio		2143702	100%
Garza Fernández Vanessa Judith		2145305	100%

González Martínez Mario Iván		2117170	100%
González Parra Sylvia Yadira		2142962	100%
López Guzmán Eder Abihú		2116464	100%
Peña Mendoza Cesar Maximiliano		2144135	100%
Pérez Ordaz Andrés Daniel		2163200	100%
Salazar Torres Isaias Leonardo		2227016	100%

Fecha: 15/09/2026

Índice

Introducción 6

Tipos de Memorias y Sistemas de Archivos 7

- Memoria Física 7
- Memoria Virtual 8
- Requisitos de la Gestión de Memoria. 9
  - Reubicación 9
  - Protección 10
  - Compartición 11
  - Organización lógica 12
  - Organización Física 13
- Particionamiento de la memoria. 14
  - Particionamiento Fijo 14
  - Particionamiento Dinámico 17
- Paginación. 20
- Segmentación 21
- Tecnicas de la gestión de memoria 22
- Sistemas de Escritorio 23
- Sistemas Móviles 26
- Sistemas Operativos de Red 28

Importancia de la Administración de Memoria 29

Control de los Espacios Disponibles y Ocupados en Memoria 30

Problemas Frecuentes en la Administración de Memoria 31

Conclusión General 35

Conclusiones Individuales 36

Bibliografía 48

Índice de Imágenes

Imagen 1. Sistemas de Escritorio. 7
Imagen 2. Sistemas Móviles. 8
Imagen 3. Memoria Física y Virtual 9
Imagen 4. Requisitos de direccionamiento para un proceso. 10
Imagen 5. Ejemplo de particionamiento fijo..... 15
Imagen 6. Esquema del proceso de paginación bajo demanda y ajuste del conjunto de trabajo en Windows. 24
Imagen 7. Estructura de la Tabla Maestra de Archivos (MFT) y organización de clústeres en el sistema de archivos NTFS. 24
Imagen 8. Organización en grupos de cilindros del Fast File System. 25
Imagen 9. Funcionamiento del buddy system para la asignación de bloques de páginas físicamente contiguas en el núcleo de Linux. 26
Imagen 10. Fragmentación Interna. 32
Imagen 11. Fragmentación Externa. 33
Imagen 12. Fallo de página. 34
Imagen 13. Thrashing..... 34

Introducción

En esta Actividad Fundamental se presenta una investigación enfocada en dos componentes esenciales de los Sistemas operativos: la gestión de memoria y el manejo de archivos.

El objetivo de este trabajo es comprender de manera clara cómo un sistema operativo organiza la información almacenada y de qué forma administra la memoria disponible para que las aplicaciones funcionen correctamente.

A lo largo de este documento, se analizan los distintos tipos de memoria y los sistemas de archivos que implementan los sistemas operativos más representativos.

Así mismo, se aborda acerca de la importancia de la administración de la memoria en el rendimiento de los dispositivos y en la capacidad de realizar varias tareas a la vez de forma fluida.

Concluyendo con un análisis de los problemas y fallos más frecuentes que surgen durante el proceso de la administración de memoria.

Tipos de Memorias y Sistemas de Archivos

Memoria Física

La memoria física de cualquier computadora es un recurso limitado, y los sistemas operativos modernos, tanto los de escritorio y servidor como los móviles, resuelven ese problema mediante memoria virtual paginada bajo demanda. Cada proceso trabaja con un espacio de direcciones lógico que puede ser mucho mayor que la memoria real disponible, y el sistema operativo se encarga de traer a la memoria principal solo las páginas necesarias en cada momento, apoyándose en una tabla de páginas por proceso y en un algoritmo de reemplazo cuando la memoria se agota.

En sistemas de escritorio: Ese algoritmo casi siempre es una variante del algoritmo del reloj, porque ofrece un buen equilibrio entre aproximarse al rendimiento de LRU y mantener baja la sobrecarga de cómputo.



Imagen 1. Sistemas de Escritorio.

En sistemas móviles: El problema cambia de naturaleza. Como la memoria flash se desgasta con cada escritura, ninguno de los sistemas móviles principales recurre a una partición de intercambio tradicional en disco; en su lugar comprimen páginas dentro de la propia RAM y, si eso no basta, terminan procesos completos.

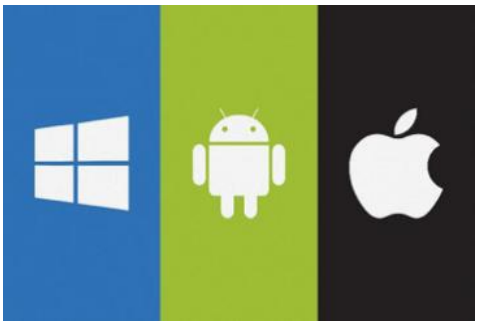


Imagen 2. Sistemas Móviles.

En cuanto al almacenamiento persistente, la idea de fondo se repite en todas las plataformas: un archivo se representa mediante una **estructura de control con metadatos** —propietario, permisos, tamaño, fechas, punteros a los bloques de datos— y los directorios se organizan jerárquicamente como listas de nombres de archivo asociadas a esas estructuras. Lo que cambia entre sistemas es el formato concreto en disco y las optimizaciones específicas para cada tipo de almacenamiento físico (disco magnético, SSD o memoria flash NAND).

Memoria Virtual

La memoria virtual es importante porque fue concebida como un método para tener múltiples trabajos de usuario residiendo en memoria principal de forma concurrente, de forma que no exista un intervalo de tiempo de espera en la ejecución de procesos sucesivos. Además, mediante el sistema de paginación y el hardware de proyección dinámica, se logra relacionar las direcciones virtuales utilizadas en el programa y una dirección real, o dirección física, de memoria principal", permitiendo que el diseñador desarrolle un mecanismo de traducción de direcciones que genere poca sobrecarga y una política de asignación de almacenamiento que minimice el tráfico entre los diferentes niveles de la jerarquía de memoria.

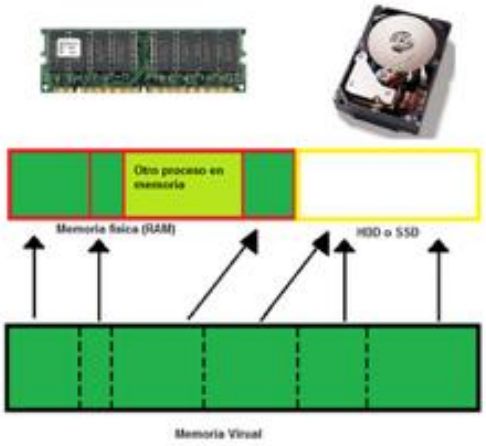


Imagen 3. Memoria Física y Virtual

Requisitos de la Gestión de Memoria.

Mientras se analizan arios mecanismos y políticas asociados con la gestión de la memoria, es útil mantener en mente los requisitos que la gestión de la memoria debe satisfacer sugiere cinco requisitos:

- **Reubicación.**
- **Protección.**
- **Compartición.**
- **Organización lógica.**
- **Organización física.**

Reubicación

En un sistema multiprogramado, la memoria principal se comparte entre varios procesos, y el programador no puede saber anticipadamente qué programas residirán en memoria principal en tiempo de ejecución. Sería bueno poder intercambiar procesos en la memoria principal para maximizar la utilización del procesador. Una vez que un programa se ha llevado al disco, podría ser necesario reubicar el proceso a un área de memoria diferente al traerlo de nuevo.

Por tanto, no se puede conocer de forma anticipada dónde se va a colocar un programa, y se debe permitir que los programas se puedan mover en la memoria principal, debido al intercambio o swap. Se asumirá que la imagen de un proceso ocupa una región contigua de la memoria principal. El sistema operativo necesitará conocer la ubicación de la información de control del proceso, de la pila de ejecución y el punto de entrada del proceso; estas direcciones son fáciles de adquirir porque el sistema operativo gestiona la memoria. Sin embargo, el procesador también debe tratar con referencias de memoria dentro del propio programa: las instrucciones de salto contienen una dirección de la instrucción a ejecutar, y las instrucciones de referencia de datos contienen la dirección del dato referenciado. El hardware del procesador y el software del sistema operativo deben traducir estas referencias de memoria en direcciones físicas que reflejen la ubicación actual del programa en la memoria principal.

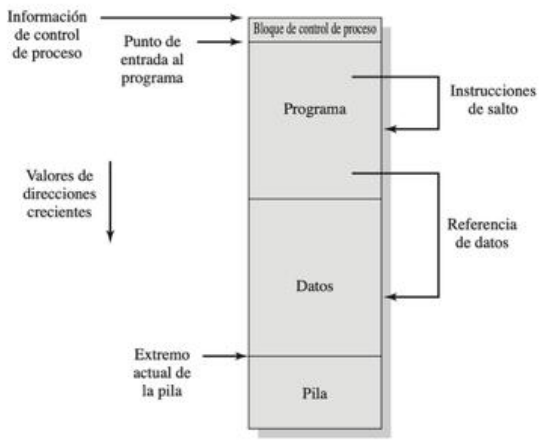


Imagen 4. Requisitos de direccionamiento para un proceso.

Protección

Cada proceso debe protegerse contra interferencias no deseadas por parte de otros procesos, accidentales o intencionadas, de modo que los programas de otros procesos no puedan referenciar sin permiso posiciones de memoria de un proceso, tanto en lectura como en escritura. Lograr los requisitos de reubicación incrementa la dificultad de satisfacer los requisitos de protección. Además, la

mayoría de los lenguajes de programación permite el cálculo dinámico de direcciones en tiempo de ejecución, por lo que todas las referencias de memoria generadas por un proceso deben comprobarse en tiempo de ejecución para asegurar que se refieren sólo al espacio de memoria asignado a dicho proceso. Los mecanismos que dan soporte a la reasignación también dan soporte al requisito de protección.

Normalmente, un proceso de usuario no puede acceder a ninguna porción del sistema operativo, ni al código ni a los datos, y un programa de un proceso no puede saltar a una instrucción de otro proceso ni, sin un trato especial, acceder al área de datos de otro proceso. El procesador debe ser capaz de abortar tales instrucciones en el punto de ejecución.

Los requisitos de protección de memoria deben ser satisfechos por el procesador (hardware) en lugar del sistema operativo (software), porque el sistema operativo no puede anticipar todas las referencias de memoria que hará un programa, y aunque pudiera, llevaría demasiado tiempo calcularlo para cada programa a fin de comprobar la violación de referencias de memoria. Por tanto, sólo es posible evaluar la permisibilidad de una referencia en tiempo de ejecución de la instrucción que realiza dicha referencia, y para ello el hardware del procesador debe tener esta capacidad.

Compartición

Cualquier mecanismo de protección debe tener la flexibilidad de permitir a varios procesos acceder a la misma porción de memoria principal. Por ejemplo, si varios programas están ejecutando el mismo programa, es ventajoso permitir que cada proceso pueda acceder a la misma copia del programa en lugar de tener su propia copia separada. Procesos que estén cooperando en la misma tarea podrían ne- cesitar compartir el acceso a la misma estructura de datos. Por tanto, el sistema de gestión de la me- moria debe permitir el acceso controlado a áreas de memoria compartidas sin comprometer la protec- ción esencial. De nuevo, se verá que los

mecanismos utilizados para dar soporte a la reubicación soportan también capacidades para la compartición.

Organización lógica

Casi invariablemente, la memoria principal de un computador se organiza como un espacio de alma- cenamiento lineal o unidimensional, compuesto por una secuencia de bytes o palabras. A nivel físico, la memoria secundaria está organizada de forma similar. Mientras que esta organización es similar al hardware real de la máquina, no se corresponde a la forma en la cual los programas se construyen normalmente. La mayoría de los programas se organizan en módulos, algunos de los cuales no se pueden modificar (sólo lectura, sólo ejecución) y algunos de los cuales contienen datos que se pueden modificar. Si el sistema operativo y el hardware del computador pueden tratar de forma efectiva los programas de usuarios y los datos en la forma de módulos de algún tipo, entonces se pueden lograr varias ventajas:

1. Los módulos se pueden escribir y compilar independientemente, con todas las referencias de un módulo desde otro resueltas por el sistema en tiempo de ejecución.
2. Con una sobrecarga adicional modesta, se puede proporcionar diferentes grados de protección a los módulos (sólo lectura, sólo ejecución).
3. Es posible introducir mecanismos por los cuales los módulos se pueden compartir entre los procesos. La ventaja de proporcionar compartición a nivel de módulo es que se corresponde con la forma en la que el usuario ve el problema, y por tanto es fácil para éste especificar la compartición deseada.

La herramienta que más adecuadamente satisface estos requisitos es la segmentación, que es una de las técnicas de gestión de la memoria exploradas en este capítulo.

Organización Física

La memoria del computador se organiza en al menos dos niveles: memoria principal y memoria secundaria. La memoria principal proporciona acceso rápido a un coste relativamente alto y es volátil, es decir, no proporciona almacenamiento permanente. La memoria secundaria es más lenta y más barata, y normalmente no es volátil. Por tanto, la memoria secundaria de larga capacidad puede proporcionar almacenamiento para programas y datos a largo plazo, mientras que una memoria principal más pequeña contiene programas y datos actualmente en uso.

En este esquema de dos niveles, la organización del flujo de información entre la memoria principal y secundaria es una de las preocupaciones principales del sistema. Esta responsabilidad podría asignarse a cada programador, pero no es practicable ni deseable por dos motivos:

La memoria principal disponible para un programa más sus datos podría ser insuficiente, en cuyo caso el programador debería utilizar la técnica de superposición (overlaying), en la que los programas y datos se organizan para asignar la misma región de memoria a varios módulos, con un programa principal responsable de intercambiar los módulos entre disco y memoria según las necesidades. Incluso con herramientas de compilación, programar con overlays malgasta tiempo del programador.

En un entorno multiprogramado, el programador no conoce en tiempo de codificación cuánto espacio estará disponible ni dónde se localizará.

Por tanto, la tarea de mover la información entre los dos niveles de memoria debería ser una responsabilidad del sistema, y esta tarea es la esencia de la gestión de la memoria.

Particionamiento de la memoria.

La operación principal de la gestión de la memoria es traer los procesos a la memoria principal para que el procesador los pueda ejecutar. En casi todos los sistemas multiprogramados modernos, esto implica el uso de un esquema sofisticado denominado memoria virtual. La memoria virtual se basa en una o ambas de las siguientes técnicas básicas: segmentación y paginación. Antes de fijarse en estas técnicas de memoria virtual, se debe preparar el camino, analizando técnicas más sencillas que no utilizan memoria virtual. Una de estas técnicas, el particionamiento, se ha utilizado en algunas variantes de ciertos sistemas operativos ahora obsoletos. Las otras dos técnicas, paginación sencilla y segmentación sencilla, no son utilizadas de forma aislada. Sin embargo, quedará más clara la discusión de la memoria virtual si se analizan primero estas dos técnicas sin tener en cuenta consideraciones de memoria virtual.

Particionamiento Fijo.

En la mayoría de los esquemas para gestión de la memoria, se puede asumir que el sistema operativo ocupa alguna porción fija de la memoria principal y que el resto de la memoria principal está disponi- ble para múltiples procesos. El esquema más simple para gestionar la memoria disponible es repartir- la en regiones con límites fijos.

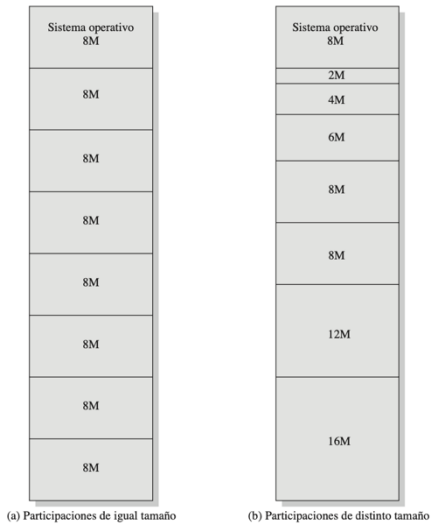


Imagen 5. Ejemplo de particionamiento fijo

En este caso, cualquier proceso cuyo tamaño es menor o igual que el tamaño de partición puede cargarse en cualquier partición disponible. Si to- das las particiones están llenas y no hay ningún proceso en estado Listo o Ejecutando, el sistema ope- rativo puede mandar a swap a un proceso de cualquiera de las particiones y cargar otro proceso, de forma que el procesador tenga trabajo que realizar.

Existen dos dificultades con el uso de las particiones fijas del mismo tamaño:

- Un programa podría ser demasiado grande para caber en una partición. En este caso, el pro- gramador debe diseñar el programa con el uso de overlays, de forma que sólo se necesite una porción del programa en memoria principal en un momento determinado. Cuando se ne- cesita un módulo que no está presente, el programa de usuario debe cargar dicho módulo en la partición del programa, superponiéndolo (overlaying) a cualquier programa o datos que haya allí.
- La utilización de la memoria principal es extremadamente ineficiente. Cualquier programa, sin importar lo pequeño que sea, ocupa una partición entera. En el ejemplo, podría haber un pro- grama cuya longitud es menor que 2 Mbytes; ocuparía una partición de 8 Mbytes cuando se lleva a la memoria. Este fenómeno, en el cual hay espacio interno malgastado debido al hecho de que el bloque de datos cargado es menor que la partición, se conoce con el nombre de **fragmentación interna.**

Algoritmo de ubicación.

Con particiones del mismo tamaño, la ubicación de los procesos en memoria es trivial. En cuanto haya una partición disponible, un proceso se carga en dicha partición. Debido a que todas las particiones son del mismo tamaño, no importa qué partición se utiliza. Si todas las particiones se encuentran ocupadas por procesos que no están listos para ejecutar, entonces uno de dichos procesos debe llevarse a disco para dejar espacio para un nuevo proceso. Cuál de los procesos se lleva a disco es una decisión de planificación.

Con particiones de diferente tamaño, hay dos formas posibles de asignar los procesos a las particiones. La forma más sencilla consiste en asignar cada proceso a la partición más pequeña dentro de la cual cabe. En este caso, se necesita una cola de planificación para cada partición, que mantenga procesos en disco destinados a dicha partición. La ventaja de esta técnica es que los procesos siempre se asignan de tal forma que se minimiza la memoria malgastada dentro de una partición (fragmentación interna).

Aunque esta técnica parece óptima desde el punto de vista de una partición individual, no es óptima desde el punto de vista del sistema completo. Por ejemplo, se considera un caso en el que no haya procesos con un tamaño entre 12 y 16M en un determinado instante de tiempo. En este caso, la partición de 16M quedará sin utilizarse, incluso aunque se puede asignar dicha partición a algunos procesos más pequeños. Por tanto, una técnica óptima sería emplear una única cola para todos los procesos. En el momento de cargar un proceso en la memoria principal, se selecciona la partición más pequeña disponible que puede albergar dicho proceso. Si todas las particiones están ocupadas, se debe llevar a cabo una decisión para enviar a swap a algún proceso. Tiene preferencia a la hora de ser expulsado a disco el proceso que ocupe la partición más pequeña que pueda albergar al proceso entrante. Es también posible considerar otros factores, como la prioridad o una preferencia por expulsar a disco procesos bloqueados frente a procesos listos.

El uso de particiones de distinto tamaño proporciona un grado de flexibilidad frente a las particiones fijas. Adicionalmente, se puede decir que los esquemas de

particiones fijas son relativamente sencillos y requieren un soporte mínimo por parte del sistema operativo y una sobrecarga de procesamiento mínimo. Sin embargo, tiene una serie de desventajas:

El número de particiones especificadas en tiempo de generación del sistema limita el número de procesos activos (no suspendidos) del sistema.

Debido a que los tamaños de las particiones son preestablecidos en tiempo de generación del sistema, los trabajos pequeños no utilizan el espacio de las particiones eficientemente. En un entorno donde el requisito de almacenamiento principal de todos los trabajos se conoce de antemano, esta técnica puede ser razonable, pero en la mayoría de los casos, se trata de una técnica ineficiente.

El uso de particionamiento fijo es casi desconocido hoy en día. Un ejemplo de un sistema operativo exitoso que sí utilizó esta técnica fue un sistema operativo de los primeros mainframes de IBM, el sistema operativo OS/MFT (Multiprogramming with a Fixed Number of Tasks; Multiprogramado con un número fijo de tareas).

Particionamiento Dinámico.

Para vencer algunas de las dificultades con particionamiento fijo, se desarrolló una técnica conocida como particionamiento dinámico. De nuevo, esta técnica se ha sustituido por técnicas de gestión de memoria más sofisticadas. Un sistema operativo importante que utilizó esta técnica fue el sistema operativo de mainframes de IBM, el sistema operativo OS/MVT (Multiprogramming with a Variable Number of Tasks; Multiprogramado con un número variable de tareas).

Con particionamiento dinámico, las particiones son de longitud y número variable. Cuando se lleva un proceso a la memoria principal, se le asigna exactamente tanta memoria como requiera y no más. Un ejemplo utiliza 64 Mbytes de memoria principal. Inicialmente, la memoria principal está vacía, excepto por el sistema operativo. Los primeros tres procesos se cargan justo donde el sistema

operativo finaliza y ocupando el espacio justo para cada proceso. Esto deja un «hueco» al final de la memoria que es demasiado pequeño para un cuarto proceso.

En algún momento, ninguno de los procesos que se encuentran en memoria está disponible. El sistema operativo lleva el proceso 2 al disco, que deja suficiente espacio para cargar un nuevo proceso, el proceso 4. Debido a que el proceso 4 es más pequeño que el proceso 2, se crea otro pequeño hueco. Posteriormente, se alcanza un punto en el cual ninguno de los procesos de la memoria principal está listo, pero el proceso 2, en estado Listo-Suspendido, está disponible. Porque no hay espacio suficiente en la memoria para el proceso 2, el sistema operativo lleva a disco el proceso 1 y lleva a la memoria el proceso 2.

Como muestra este ejemplo, el método comienza correctamente, pero finalmente lleva a una situación en la cual existen muchos huecos pequeños en la memoria. A medida que pasa el tiempo, la memoria se fragmenta cada vez más y la utilización de la memoria se decrementa. Este fenómeno se conoce como fragmentación externa, indicando que la memoria que es externa a todas las particiones se fragmenta de forma incremental, por contraposición a lo que ocurre con la fragmentación interna, descrita anteriormente.

Una técnica para eliminar la fragmentación externa es la compactación: de vez en cuando, el sistema operativo desplaza los procesos en memoria, de forma que se encuentren contiguos y de este modo toda la memoria libre se encontrará unida en un bloque. Por ejemplo, la compactación permite obtener un bloque de memoria libre de longitud 16M. Esto sería suficiente para cargar un proceso adicional. La desventaja de la compactación es el hecho de que se trata de un procedimiento que consume tiempo y malgasta tiempo de procesador. Obsérvese que la compactación requiere la capacidad de reubicación dinámica. Es decir, debe ser posible mover un programa desde una región a otra en la memoria principal sin invalidar las referencias de la memoria de cada programa.

Algoritmo de ubicación.

Debido a que la compactación de memoria consume una gran cantidad de tiempo, el diseñador del sistema operativo debe ser inteligente a la hora de decidir cómo asignar la memoria a los procesos (cómo eliminar los huecos). A la hora de cargar o intercambiar un proceso a la memoria principal, y siempre que haya más de un bloque de memoria libre de suficiente tamaño, el sistema operativo debe decidir qué bloque libre asignar. Tres algoritmos de colocación que pueden considerarse son mejor-ajuste (best-fit), primer-ajuste (first-fit) y siguiente-ajuste (next-fit). Todos están limitados a escoger entre los bloques libres de la memoria principal que son iguales o más grandes que el proceso que va a llevarse a la memoria. Mejor-ajuste escoge el bloque más cercano en tamaño a la petición. Primer-ajuste comienza a analizar la memoria desde el principio y escoge el primer bloque disponible que sea suficientemente grande. Siguiente-ajuste comienza a analizar la memoria desde la última colocación y elige el siguiente bloque disponible que sea suficientemente grande.

Cuál de estas técnicas es mejor depende de la secuencia exacta de intercambio de procesos y del tamaño de dichos procesos, aunque se pueden hacer algunos comentarios. El algoritmo primer-ajuste no es sólo el más sencillo, sino que normalmente es también el mejor y más rápido. El algoritmo siguiente-ajuste tiende a producir resultados ligeramente peores que el primer-ajuste, ya que llevará más frecuentemente a una asignación de un bloque libre al final de la memoria, con el resultado de que el bloque más grande de memoria libre se divide rápidamente en pequeños fragmentos; por tanto, en el caso del algoritmo siguiente-ajuste se puede requerir más frecuentemente la compactación. Por otro lado, el algoritmo primer-ajuste puede dejar el final del espacio de almacenamiento con pequeñas particiones libres que necesitan buscarse en cada paso del primer-ajuste siguiente.

El algoritmo mejor-ajuste, a pesar de su nombre, su comportamiento normalmente es el peor. Debido a que el algoritmo busca el bloque más pequeño que satisfaga la petición, garantiza que el fragmento que quede sea lo más pequeño

posible. Aunque cada petición de memoria siempre malgasta la cantidad más pequeña de la memoria, el resultado es que la memoria principal se queda rápidamente con bloques demasiado pequeños para satisfacer las peticiones de asignación de la memoria. Por tanto, la compactación de la memoria se debe realizar más frecuentemente que con el resto de los algoritmos.

Algoritmo de ubicación.

En un sistema multiprogramado que utiliza particionamiento dinámico, puede haber un momento en el que todos los procesos de la memoria principal estén en estado bloqueado y no haya suficiente me- moria para un proceso adicional, incluso después de llevar a cabo una compactación. Para evitar mal- gastar tiempo de procesador esperando a que un proceso se desbloquee, el sistema operativo inter- cambiará alguno de los procesos entre la memoria principal y disco para hacer sitio a un nuevo proceso o para un proceso que se encuentre en estado Listo-Suspendido. Por tanto, el sistema operati- vo debe escoger qué proceso reemplazar. Debido a que el tema de los algoritmos de reemplazo se contemplará en detalle respecto a varios esquemas de la memoria virtual, se pospone esta discusión hasta entonces.

Paginación.

El término memoria virtual se asocia habitualmente con sistemas que emplean paginación, a pesar de que la memoria virtual basada en segmentación también se utiliza y será tratada más adelante. El uso de paginación para conseguir memoria virtual fue utilizado por primera vez en el computador Atlas y pronto se convirtió en una estrategia usada en general de forma comercial.

En la presentación de la paginación sencilla, indicamos que cada proceso dispone de su propia tabla de páginas, y que todas las páginas se encuentran localizadas en la memoria principal. Cada entrada en la tabla de páginas consiste en un número de marco de la correspondiente página en la memoria principal. Para

la memoria virtual basada en el esquema de paginación también se necesita una tabla de páginas. De nuevo, normalmente se asocia una única tabla de páginas a cada proceso. En este caso, sin embargo, las entradas de la tabla de páginas son más complejas. Debido a que sólo algunas de las páginas de proceso se encuentran en la memoria principal, se necesita que cada entrada de la tabla de páginas indique si la correspondiente página está presente (P) en memoria principal o no. Si el bit indica que la página está en memoria, la entrada también debe indicar el número de marco de dicha página.

La entrada de la tabla de páginas incluye un bit de modificado (M), que indica si los contenidos de la correspondiente página han sido alterados desde que la página se cargó por última vez en la memoria principal. Si no había ningún cambio, no es necesario escribir la página cuando llegue el momento de reemplazarla por otra página en el marco de página que actualmente ocupa. Pueden existir también otros bits de control en estas entradas. Por ejemplo, si la protección y compartición se gestiona a nivel de página, se necesitarán también los bits para este propósito.

Segmentación

La segmentación permite al programador ver la memoria como si se tratase de diferentes espacios de direcciones o segmentos. Los segmentos pueden ser de tamaños diferentes, en realidad de tamaño dinámico. Una referencia a la memoria consiste en un formato de dirección del tipo (número de segmento, desplazamiento).

Esta organización tiene un gran número de ventajas para el programador sobre los espacios de direcciones no segmentados:

1) Simplifica el tratamiento de estructuras de datos que pueden crecer. Si el programador no conoce a priori el tamaño que una estructura de datos en particular puede alcanzar es necesario hacer una estimación salvo que se utilicen tamaños de segmento dinámicos. Con la memoria virtual segmentada, a una estructura de datos se le puede asignar su propio segmento, y el sistema operativo expandirá o reducirá el segmento bajo demanda. Si un segmento que

necesita expandirse se encuentra en la memoria principal y no hay suficiente tamaño, el sistema operativo puede mover el segmento a un área de la memoria principal mayor, si se encuentra disponible, o enviarlo a swap. En este último caso el segmento al que se ha incrementado el tamaño volverá a la memoria principal en la siguiente oportunidad que tenga.

2) Permite programas que se modifican o recopilan de forma independiente, sin requerir que el conjunto completo de programas se re-enlacen y se vuelvan a cargar. Esta posibilidad se puede articular por medio de la utilización de múltiples segmentos.
3) Da soporte a la compartición entre procesos. El programador puede situar un programa de utilidad o una tabla de datos que resulte útil en un segmento al que pueda hacerse referencia desde otros procesos.
4) Soporta los mecanismos de protección. Esto es debido a que un segmento puede definirse para contener un conjunto de programas o datos bien descritos, y el programador o el administrador de sistemas puede asignar privilegios de acceso de una forma apropiada.

Tecnicas de la gestión de memoria

Técnica	Descripción	Virtudes	Defectos
Particionamiento fijo	La memoria principal se divide en particiones estáticas en tiempo de generación del sistema. Un proceso se puede cargar en una partición con igual o superior tamaño.	Sencilla de implementar; poca sobrecarga para el sistema operativo.	Uso ineficiente de la memoria, debido a la fragmentación interna; debe fijarse el número máximo de procesos activos.
Particionamiento dinámico	Las particiones se crean de forma dinámica, de tal forma que cada proceso se carga en una partición del mismo tamaño que el proceso.	No existe fragmentación interna; uso más eficiente de memoria principal.	Uso ineficiente del procesador, debido a la necesidad de compactación para evitar la fragmentación externa.

Técnica	Descripción	Virtudes	Defectos
Paginación sencilla	La memoria principal se divide en marcos del mismo tamaño. Cada proceso se divide en páginas del mismo tamaño que los marcos. Un proceso se carga a través de la carga de todas sus páginas en marcos disponibles, no necesariamente contiguos.	No existe fragmentación externa.	Una pequeña cantidad de fragmentación interna.
Segmentación sencilla	Cada proceso se divide en segmentos. Un proceso se carga cargando todos sus segmentos en particiones dinámicas, no necesariamente contiguos.	No existe fragmentación interna; mejora la utilización de la memoria y reduce la sobrecargada respecto al particionamiento dinámico.	Fragmentación externa.
Paginación con memoria virtual	Exactamente igual que la paginación sencilla, excepto que no es necesario cargar todas las páginas de un proceso. Las páginas no residentes se traen bajo demanda de forma automática.	No existe fragmentación externa; mayor grado de multiprogramación; gran espacio de direcciones virtuales.	Sobrecarga por la gestión compleja de la memoria.
Segmentación con memoria virtual	Exactamente igual que la segmentación, excepto que no es necesario cargar todos los segmentos de un proceso. Los segmentos no residentes se traen bajo demanda de forma automática.	No existe fragmentación interna; mayor grado de multiprogramación; gran espacio de direcciones virtuales; soporte a protección y compartición.	Sobrecarga por la gestión compleja de la memoria.

Sistemas de Escritorio

Windows

En los sistemas Windows, la memoria se gestiona mediante paginación bajo demanda con clustering. Esto significa que, ante un fallo de página, el sistema no carga solo una, sino varias páginas sucesivas anticipando su uso futuro. Cada proceso recibe un conjunto de trabajo con un límite mínimo y máximo de páginas,

el cual se ajusta dinámicamente si la memoria escasea. Para liberar espacio de forma eficiente, el sistema emplea una variante del algoritmo del reloj en monoprocesadores, o el algoritmo FIFO en entornos multiprocesador.

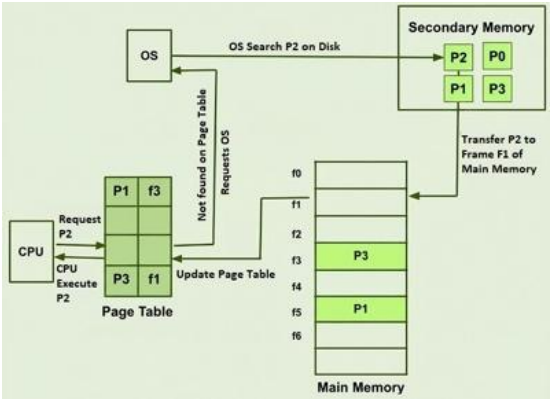


Imagen 6. Esquema del proceso de paginación bajo demanda y ajuste del conjunto de trabajo en Windows.

Para el almacenamiento local, Windows utiliza NTFS, un sistema que trata absolutamente todo como un objeto estructurado con atributos. La información de los archivos se organiza detalladamente en la Tabla Maestra de Archivos (MFT), utilizando números lógicos de clúster (LCN) para ubicar los datos físicos en el disco. Además, los directorios se indexan mediante árboles B+, una estructura que garantiza búsquedas muy rápidas sin requerir reorganizaciones constantes.

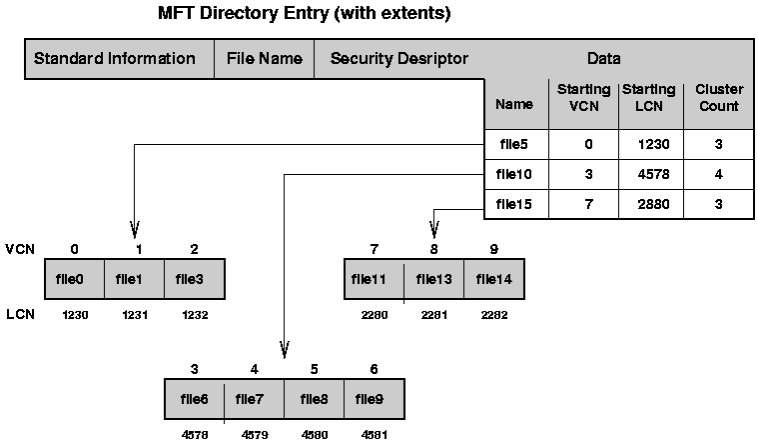


Imagen 7. Estructura de la Tabla Maestra de Archivos (MFT) y organización de clústeres en el sistema de archivos NTFS.

UNIX / Solaris

Por su parte, Solaris aborda la memoria a través del proceso pageout, que funciona como un reloj de dos manecillas: una frontal desactiva el bit de referencia y una posterior evalúa si la página debe liberarse. Este mecanismo se activa automáticamente cuando la memoria libre cae por debajo del umbral lotsfree, ajustando su velocidad de exploración según la necesidad. Si el nivel crítico se mantiene por varios segundos, el sistema toma una medida más drástica y comienza a descargar procesos completos.

A nivel de archivos, la fuerte herencia de UNIX en Solaris dicta el uso de nodos-i (inodes). Esta estructura mantiene el control y los metadatos completamente separados del nombre del archivo, lo que permite crear múltiples enlaces apuntando a los mismos datos. Para optimizar la velocidad en discos magnéticos, arquitecturas clásicas como el Fast File System dividen las particiones en grupos de cilindros, reduciendo enormemente los tiempos de búsqueda física de los cabezales.

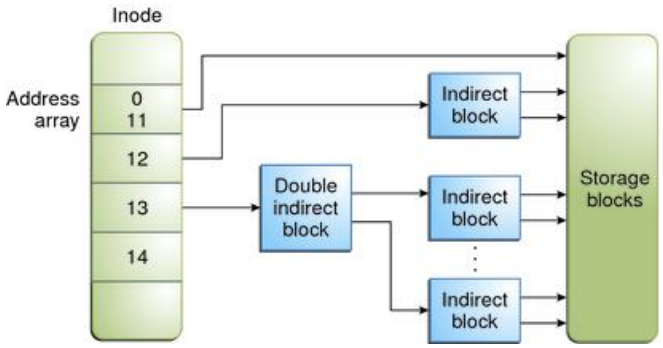


Imagen 8. Organización en grupos de cilindros del Fast File System (FFS).

LINUX

En el caso de Linux, la memoria física se organiza en zonas muy específicas para respetar las limitaciones de acceso directo (DMA) de ciertos dispositivos de

hardware. Para gestionar los bloques de páginas físicamente contiguas, el asignador del núcleo utiliza un sistema de descomposición binaria conocido como buddy system. Paralelamente, para la memoria dinámica de vida corta, el sistema emplea una asignación por franjas que recicla de forma constante estructuras y descriptores previamente liberados.

Finalmente, en cuanto al almacenamiento, Linux destaca por implementar el Sistema de Archivos Virtual (VFS). Esta capa de abstracción orientada a objetos le permite interactuar con múltiples formatos de disco o de red de forma completamente transparente para el usuario. Gracias a esta arquitectura, el sistema puede cachear nodos-i en memoria de manera unificada, acelerando los accesos futuros sin importar si el archivo subyacente usa el formato nativo EXT2 o algún sistema más moderno estructurado en bitácora.

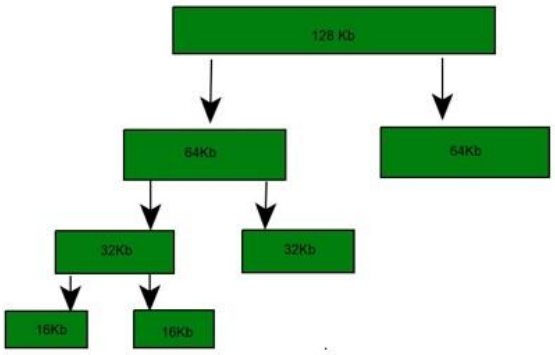


Imagen 9. Funcionamiento del buddy system para la asignación de bloques de páginas físicamente contiguas en el núcleo de Linux.

Sistemas Móviles

Android

En el ecosistema móvil, Android hereda la paginación de Linux, pero evita la partición de intercambio tradicional para no acortar la vida útil de la memoria flash. En su lugar, utiliza zRAM para comprimir las páginas inactivas directamente en la memoria RAM. Además, su entorno de ejecución ART gestiona la memoria activa mediante un recolector de basura, y cuenta con mecanismos escalonados como el

Low Memory Killer (LMK) para cerrar procesos en segundo plano cuando los recursos realmente escasean.

Para el almacenamiento, Android soporta múltiples formatos, siendo ext4 el estándar tradicional. Sin embargo, destaca la adopción progresiva de F2FS, un sistema de archivos diseñado desde cero para convivir con la naturaleza de las memorias flash. Este sistema opera escribiendo los datos de forma secuencial al final de una cola, lo que reduce significativamente la sobrecarga de escritura y mejora el rendimiento general del dispositivo.

iOS

Por su parte, iOS aborda la gestión de recursos de forma muy distinta, manejando la memoria a través del recuento automático de referencias (ARC). Esto le permite destruir y liberar objetos inmediatamente cuando dejan de usarse, sin depender de un recolector de basura en segundo plano. Ante la escasez crítica de RAM, recurre a la compresión de memoria y al proceso Jetsam, un hilo del núcleo que cierra aplicaciones completas al instante, dado que iOS no suele apoyarse en el intercambio de memoria en el disco.

A nivel de almacenamiento local, el ecosistema de Apple está gobernado por APFS, un sistema altamente optimizado para unidades de estado sólido. Sus características estrella son los clones, que evitan duplicar el peso de los datos físicos en el disco hasta que estos sufren modificaciones reales, y las instantáneas, que toman registros de solo lectura de todo el sistema para facilitar restauraciones seguras y automáticas durante las actualizaciones.

HarmonyOS

Finalmente, HarmonyOS resalta por su arquitectura modular y su evolución hacia la independencia total. En su versión más reciente, NEXT, abandonó los núcleos heredados para utilizar el microkernel propio HongMeng, el cual mantiene centralizadas solo las funciones más esenciales. Para proteger los datos, su sistema de archivos incorpora cifrado nativo por hardware mediante TrustZone y se ejecuta

de forma aislada en el espacio de usuario, garantizando que un fallo en los archivos no comprometa al sistema operativo entero.

Sistemas Operativos de Red

Los sistemas operativos de red operan bajo un paradigma totalmente diferente, ya que no se encargan de gestionar la memoria o los archivos físicos de un equipo local. En cambio, funcionan como una capa de software superpuesta a los sistemas operativos tradicionales, permitiendo que computadoras independientes se conecten entre sí para acceder a recursos compartidos, tales como bases de datos, directorios remotos o impresoras alojadas en un servidor.

La característica fundamental de este modelo es la transparencia limitada: cada nodo conectado a la red conserva intacta su propia identidad, y el usuario es plenamente consciente de que está interactuando con múltiples máquinas a la vez. Este enfoque contrasta frontalmente con la filosofía de los sistemas operativos distribuidos, un concepto clásico cuyo objetivo era ocultar por completo la complejidad de la red para presentar todas las computadoras como si fueran un único equipo centralizado.

Sin embargo, la ambiciosa idea del sistema distribuido puro terminó perdiendo protagonismo en la industria. En su lugar, el mercado adoptó el uso de "middleware", una solución mucho más pragmática y flexible. Tecnologías como CORBA o el DCOM de Microsoft demostraron que era más eficiente instalar una capa de integración sobre los sistemas operativos que ya existían, logrando conectar los equipos de manera robusta sin necesidad de reescribir toda la arquitectura del sistema.

Importancia de la Administración de Memoria

La administración de memoria es importante porque los gestores de sistema necesitan un control inherente y ordenado de la asignación de los recursos para satisfacer las necesidades de un entorno de computación que permita programación modular y el uso flexible de los datos.

Para lograrlo, el sistema operativo tiene cinco responsabilidades principales de gestión de almacenamiento:

- **Aislamiento de procesos:** El sistema operativo debe evitar que los procesos independientes interfieran en la memoria de otro proceso, tanto datos como instrucciones.
- **Asignación y gestión automática:** Permite incrementar la eficiencia asignando memoria a los trabajos solo cuando se necesiten" de forma "transparente al programador.
- **Soporte a la programación modular:** Los programadores deben ser capaces de definir módulos de programación y crear, destruir, y alterar el tamaño de los módulos dinámicamente.
- **Protección y control de acceso:** El sistema operativo debe permitir que varios usuarios puedan acceder de distintas formas a porciones de memoria, protegiendo contra características que amenazan la integridad de los programas e incluso del propio sistema operativo.
- **Almacenamiento a largo plazo:** Permite almacenar la información durante largos periodos de tiempo, después de que el computador se haya apagado a través del uso de la memoria virtual y objetos denominados ficheros.

Control de los Espacios Disponibles y Ocupados en Memoria

Se utilizan principalmente dos maneras de administrar la memoria, el mapa de bits y las listas libres.

Con un mapa de bits, la memoria se divide desde unidades de asignación tan pequeñas como unas cuantas palabras hasta varios kilobytes. A cada una de estas unidades se le asigna un bit dentro del mapa. El valor del bit permite conocer su estado: 0 indica que la unidad está libre, mientras que 1 señala que se encuentra ocupada. Por esto mismo, la capacidad de la unidad de asignación es algo vital a la hora de pensar en el diseño, pues, entre más pequeña sea la unidad de asignación, mayor será el mapa de bits.

Gracias a el mapa de bits se tiene una manera simple de llevar el registro de las palabras de memoria en una cantidad fija de memoria, pues el tamaño del mapa de bits sólo depende del tamaño de la memoria y el tamaño de la unidad de asignación.

La otra manera de llevar el registro de la memoria que se mencionó es mantener una lista ligada de segmentos de memoria asignados y libres, en donde un segmento contiene un proceso o es un hueco vacío entre dos procesos.

Se recomienda mantener la lista de segmentos ordenada por dirección, pues, ya que, al ordenarla de esta manera, si se termina un proceso o este se intercambia, el proceso de actualizar se vuelve más sencillo. Si los procesos y huecos se mantienen en una lista ordenada por dirección, se podrán utilizar varios algoritmos para asignar memoria a un proceso creado (o a un proceso existente que se intercambie del disco).

Ahora, el administrador de memoria debe ser capaz de saber cuánta memoria debe asignar, para administrarla se diferencian cuatro algoritmos principales:

Primer ajuste. Este algoritmo se caracteriza por ser rápido debido a que busca lo menos posible. En este el administrador de memoria explora la lista de segmentos hasta encontrar un hueco que sea lo bastante grande para dividirlo en dos partes, una para el proceso y otra para la memoria sin utilizar (en caso de que sea un ajuste exacto, no se dividiría).

Siguiente ajuste. Lo que diferencia a este ajuste del primero, es que este contiene un registro de dónde se encuentra cada vez que se encuentra un hueco adecuado. Si se le pide encontrar un hueco, este algoritmo empezará a buscar desde el lugar donde se había quedado.

Mejor ajuste. Este algoritmo busca en toda la lista para tomar el hueco más pequeño que funcione (por esto es más lento que el del primer ajuste), la ventaja de este es que gracias a buscar el hueco mínimo que más coincida, los huecos más grandes no se dividen, por si luego se requieren.

Peor ajuste. Este es el caso contrario que el mejor ajuste, pues siempre seleccionará el hueco más grande disponible, para que sea útil este nuevo hueco, obviamente esta opción no suele ser la óptima.

Estos algoritmos pueden ser acelerados manteniendo listas separadas para los procesos y los huecos, estos dedican toda su energía a inspeccionar los huecos, no los procesos.

Problemas Frecuentes en la Administración de Memoria

La administración de memoria es importante porque permite que el sistema operativo controle de manera ordenada la memoria disponible y pueda mantener varios procesos funcionando al mismo tiempo. Para esto, el sistema operativo debe saber qué espacios están ocupados y cuáles están disponibles, además de decidir cómo asignarlos para aprovechar mejor la memoria y evitar que el procesador tenga que esperar. Stallings explica que la gestión de memoria debe cumplir con

diferentes necesidades, como la reubicación de los procesos, la protección entre ellos, la posibilidad de compartir información y una buena organización de la memoria. Sin embargo, las diferentes formas de administrar estos recursos también pueden presentar algunos problemas.

Uno de los problemas más comunes es la **fragmentación interna**. Esta ocurre principalmente en las particiones fijas, cuando un proceso necesita menos espacio del que se le asignó. En este caso, una parte de la partición queda sin utilizar y no puede ser aprovechada por otro proceso. Aunque el espacio que se pierde en cada proceso pueda parecer pequeño, cuando hay muchos procesos puede terminar representando una cantidad importante de memoria desperdiciada.

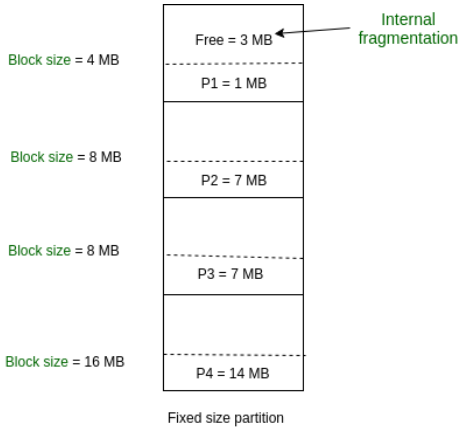


Imagen 10. Fragmentación Interna.

Otro problema es la **fragmentación externa**, que aparece principalmente en el particionamiento dinámico. Conforme los procesos entran y salen de la memoria, van quedando pequeños espacios libres entre las partes que siguen ocupadas. El problema es que estos espacios pueden quedar separados y, aunque en total exista suficiente memoria libre, puede no haber un espacio continuo suficientemente grande para colocar un nuevo proceso.

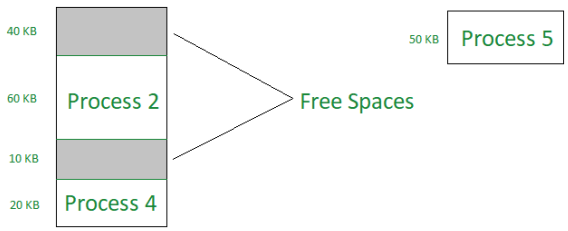


Imagen 11. Fragmentación Externa.

También existe el problema de **decidir dónde colocar cada proceso** dentro de la memoria disponible. Se presenta tres estrategias para realizar esta tarea: First-fit, Best-fit y Next-fit. Cada una busca aprovechar los espacios disponibles de diferente manera, pero también puede generar ciertos problemas. Por ejemplo, Best-fit busca utilizar el espacio libre más pequeño que pueda contener al proceso, pero esto puede dejar muchos espacios pequeños que después sean difíciles de utilizar.

Una forma de solucionar la fragmentación externa es la **compactación**, que consiste en mover los procesos para juntar los espacios libres y formar un bloque de memoria más grande. Aunque esto permite aprovechar espacios que antes estaban separados, también tiene un costo para el sistema, ya que mover los procesos y reorganizar la memoria requiere tiempo de procesamiento.

Otro grupo de problemas aparece con la **memoria virtual**. Esta permite que los procesos trabajen con un espacio de direcciones mayor que la memoria principal disponible, pero para hacerlo el sistema operativo debe controlar constantemente qué páginas se encuentran en memoria y cuáles permanecen en el almacenamiento secundario. Cuando un proceso intenta acceder a una página que no se encuentra en la memoria principal, se produce un **fallo de página**, por lo que el sistema operativo debe buscarla y cargarla en memoria para continuar con la ejecución.

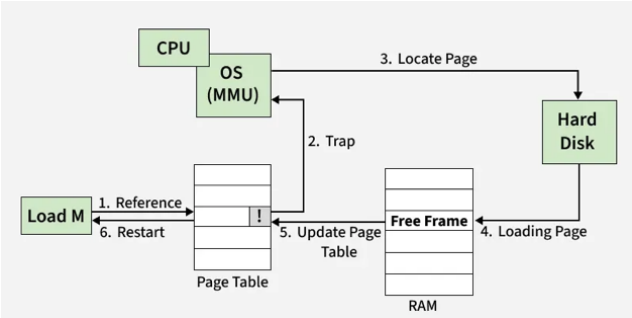


Imagen 12. Fallo de página.

Cuando los fallos de página ocurren de manera excesiva puede aparecer un problema más grave conocido como **thrashing o trasiego**. Esto sucede cuando el sistema pasa gran parte de su tiempo enviando y trayendo páginas entre la memoria principal y el almacenamiento secundario, en lugar de utilizar ese tiempo para ejecutar los procesos. Esto puede suceder cuando los procesos necesitan constantemente páginas que no se encuentran disponibles en la memoria principal. Como consecuencia, el procesador pierde gran parte del tiempo en estas operaciones y el rendimiento general del sistema disminuye considerablemente.

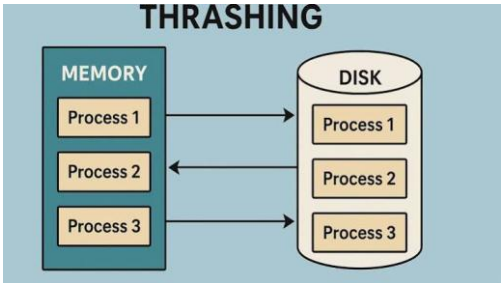


Imagen 13. Thrashing.

Conclusión General

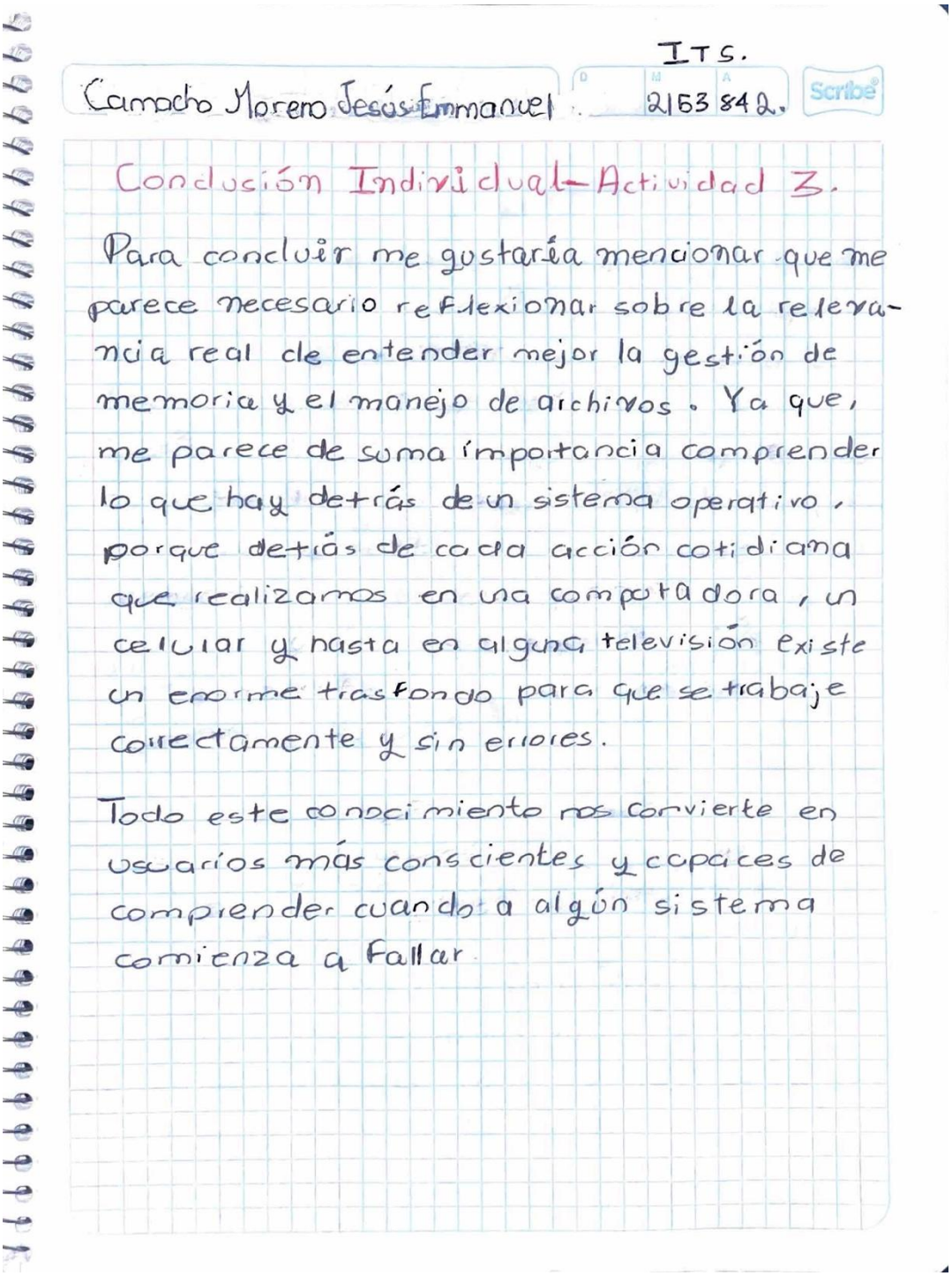
A lo largo de esta actividad se abordaron dos de los pilares fundamentales en el funcionamiento de cualquier sistema operativo: la gestión de memoria y el manejo de archivos. Ambos procesos, aunque distintos en su naturaleza, están profundamente interrelacionados, ya que la eficiencia con la que un sistema administra sus recursos de almacenamiento determina directamente el rendimiento, la estabilidad y la capacidad de respuesta de cualquier dispositivo, sea una computadora de escritorio, un servidor, un teléfono móvil o un sistema conectado en red.

Se pudo comprender que la memoria física es un recurso limitado que los sistemas operativos modernos resuelven mediante el uso de memoria virtual, permitiendo que los procesos trabajen con espacios de direcciones mayores a los realmente disponibles. Técnicas como el particionamiento fijo y dinámico, la paginación y la segmentación representan distintas soluciones históricas y actuales a un mismo problema: aprovechar al máximo la memoria principal minimizando el desperdicio de espacio y la sobrecarga de procesamiento. También quedó claro que cada plataforma adapta estas técnicas según sus propias limitaciones de hardware; mientras que los sistemas de escritorio como Windows, Linux y UNIX/Solaris priorizan el rendimiento mediante algoritmos de reemplazo de página y sistemas de archivos robustos (NTFS, FFS, VFS), los sistemas móviles como Android, iOS y HarmonyOS evitan el desgaste de la memoria flash recurriendo a la compresión en RAM y al cierre de procesos en lugar del intercambio tradicional a disco.

Asimismo, se identificó que la correcta administración de los espacios disponibles y ocupados (mediante mapas de bits o listas ligadas) es esencial para que el sistema pueda tomar decisiones de asignación eficientes, apoyándose en algoritmos como primer ajuste, siguiente ajuste, mejor ajuste y peor ajuste, cada uno con ventajas y desventajas particulares según el contexto de uso.

Conclusiones Individuales

Camacho Moreno Jesús Emmanuel



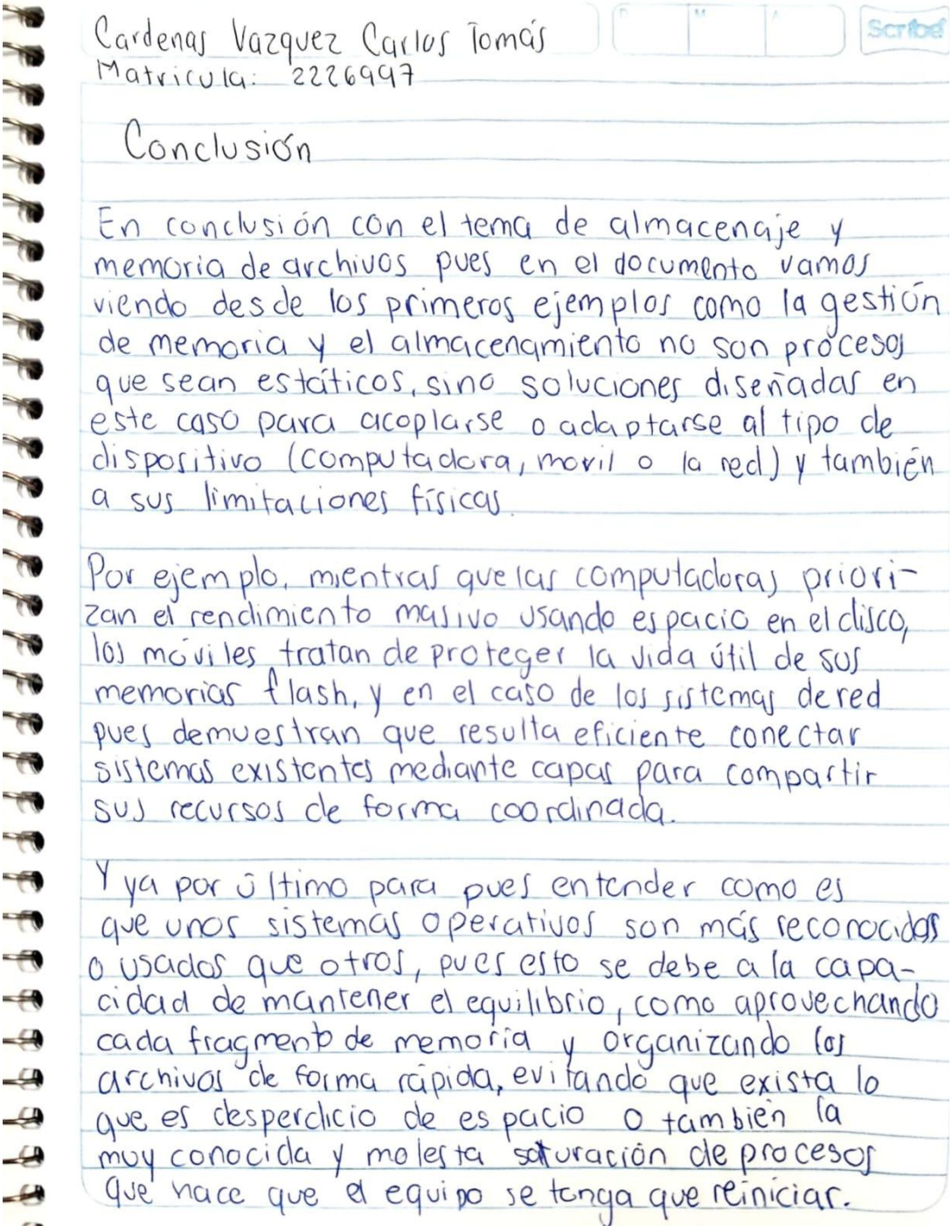
ITS.

Camacho Moreno Jesús Emmanuel D 21 M 03 A 842.

Conclusión Individual - Actividad 3.

Para concluir me gustaría mencionar que me parece necesario reflexionar sobre la relevancia real de entender mejor la gestión de memoria y el manejo de archivos. Ya que, me parece de suma importancia comprender lo que hay detrás de un sistema operativo, porque detrás de cada acción cotidiana que realizamos en una computadora, un celular y hasta en alguna televisión existe un enorme trasfondo para que se trabaje correctamente y sin errores.

Todo este conocimiento nos convierte en usuarios más conscientes y capaces de comprender cuando a algún sistema comienza a fallar.



Cardenas Vazquez Carlos Tomás
Matricula: 2226997

Conclusión

En conclusión con el tema de almacenaje y memoria de archivos pues en el documento vamos viendo desde los primeros ejemplos como la gestión de memoria y el almacenamiento no son procesos que sean estáticos, sino soluciones diseñadas en este caso para acoplarse o adaptarse al tipo de dispositivo (computadora, movil o la red) y también a sus limitaciones físicas.

Por ejemplo, mientras que las computadoras priorizan el rendimiento masivo usando espacio en el disco, los móviles tratan de proteger la vida útil de sus memorias flash, y en el caso de los sistemas de red pues demuestran que resulta eficiente conectar sistemas existentes mediante capas para compartir sus recursos de forma coordinada.

Y ya por último para pues entender como es que unos sistemas operativos son más reconocidos o usados que otros, pues esto se debe a la capacidad de mantener el equilibrio, como aprovechando cada fragmento de memoria y organizando los archivos de forma rápida, evitando que exista lo que es desperdicio de espacio o también la muy conocida y molesta saturación de procesos que hace que el equipo se tenga que reiniciar.

Juan Kristof Cerda Herrera

Conclusión Individual

Se logro analizar que el almacenamiento de memoria y archivos no es un tema aislado, sino la columna vertebral sobre la que se sostiene el funcionamiento de cualquier sistema operativo moderno.

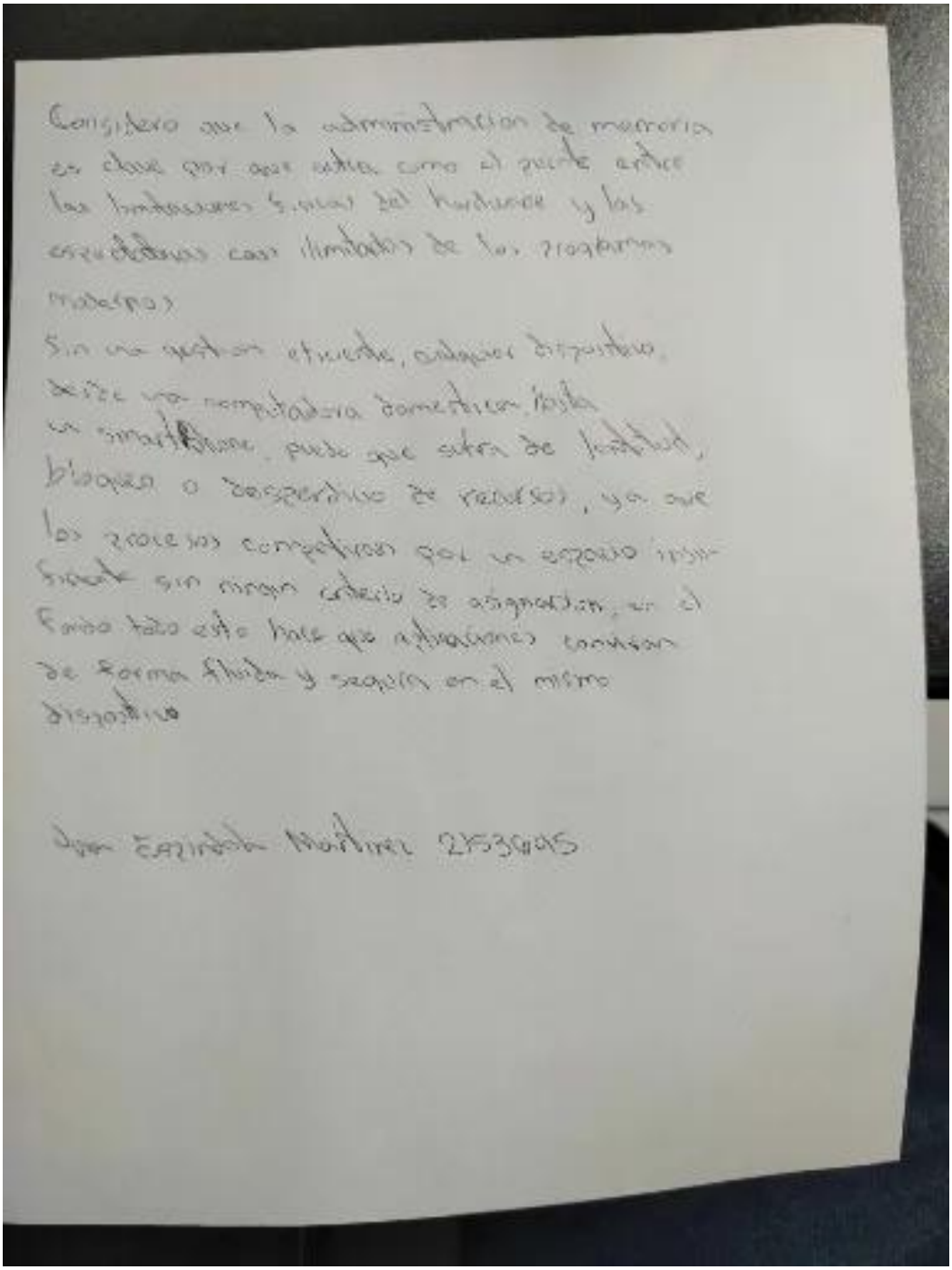
La gestión de memoria permite que multiples procesos convivan de forma segura y eficiente dentro de un recurso que, aunque cada vez más grande, sigue siendo ilimitado.

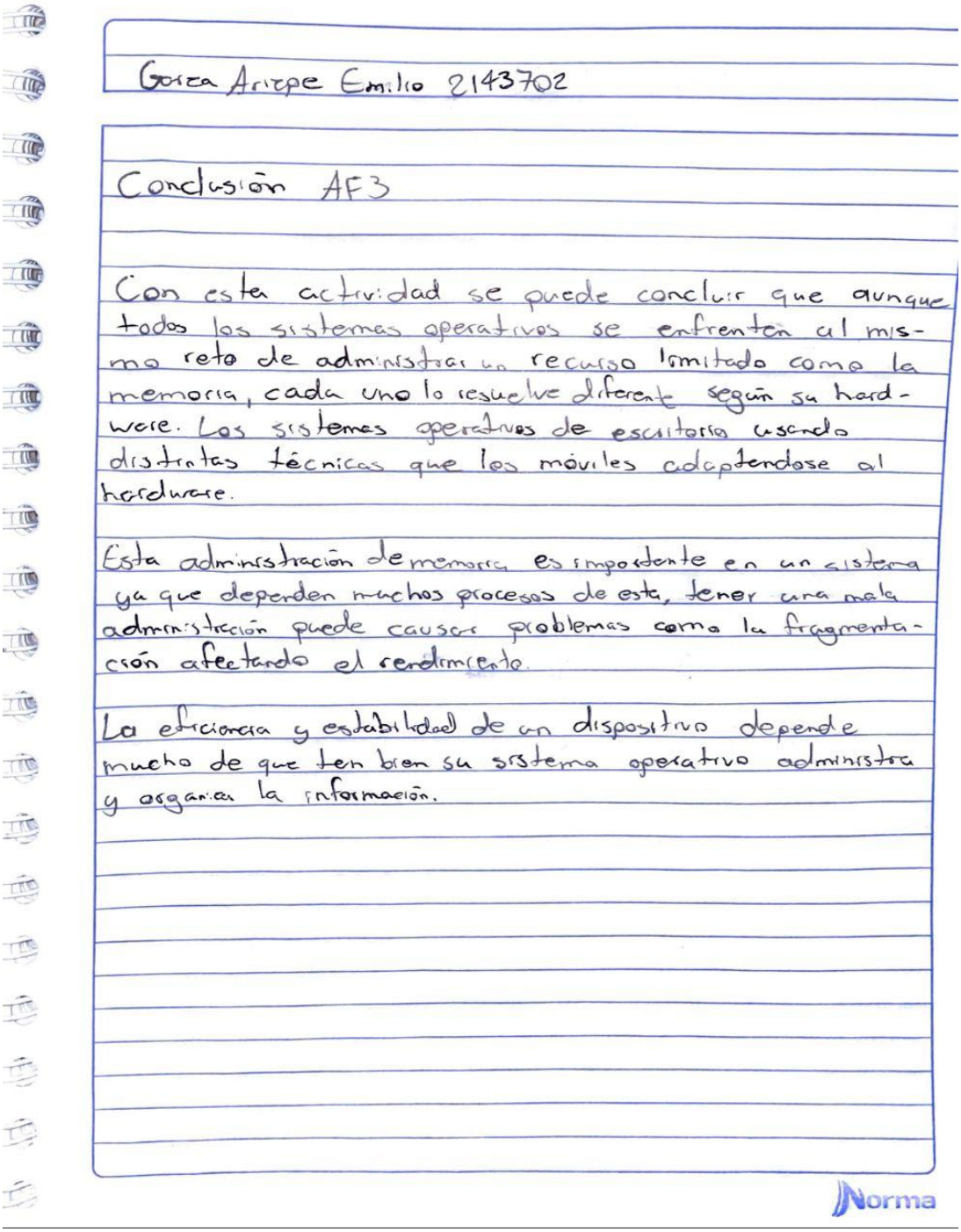
Vimos como los algoritmos de asignación buscan un equilibrio entre velocidad de asignacion y aprovechamiento del espacio, y cómo problemas como la fragmentación interna y externa son consecuencias casi inevitables de cualquier esquema de asignación, ya sea por bloques fijos o por particiones variables.

La memoria virtual, apoyada en tecnicas como la paginación por demanda, resuelve gran parte de estas limitaciones al permitir que los procesos "crean" tener más memoria de la que fisicamente existe, trasladando el problema al disco cuando sea necesario.

La administración de archivos complementan este esquema al dar persistencia a la información.

Espindola Martínez Juan





Garza Arizpe Emilio 2143702

Conclusión AF3

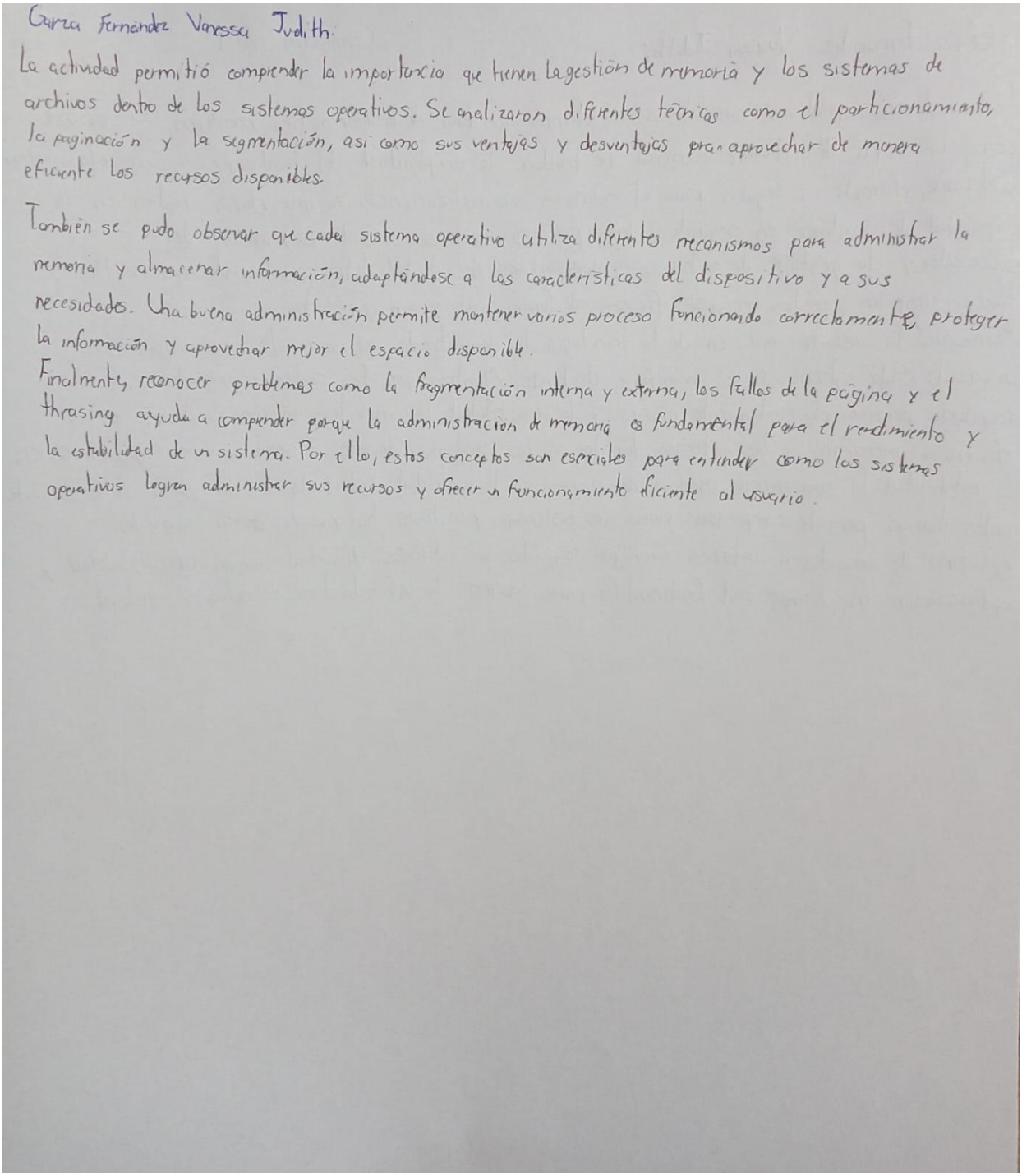
Con esta actividad se puede concluir que aunque todos los sistemas operativos se enfrentan al mismo reto de administrar un recurso limitado como la memoria, cada uno lo resuelve diferente según su hardware. Los sistemas operativos de escritorio usando distintas técnicas que los móviles adaptandose al hardware.

Esta administración de memoria es importante en un sistema ya que dependen muchos procesos de este, tener una mala administración puede causar problemas como la fragmentación afectando el rendimiento.

La eficiencia y estabilidad de un dispositivo depende mucho de que tan bien su sistema operativo administra y organiza la información.

Norma

Garza Fernández Vanessa Judith


Garza Fernández Vanessa Judith.

La actividad permitió comprender la importancia que tienen la gestión de memoria y los sistemas de archivos dentro de los sistemas operativos. Se analizaron diferentes técnicas como el particionamiento, la paginación y la segmentación, así como sus ventajas y desventajas para aprovechar de manera eficiente los recursos disponibles.

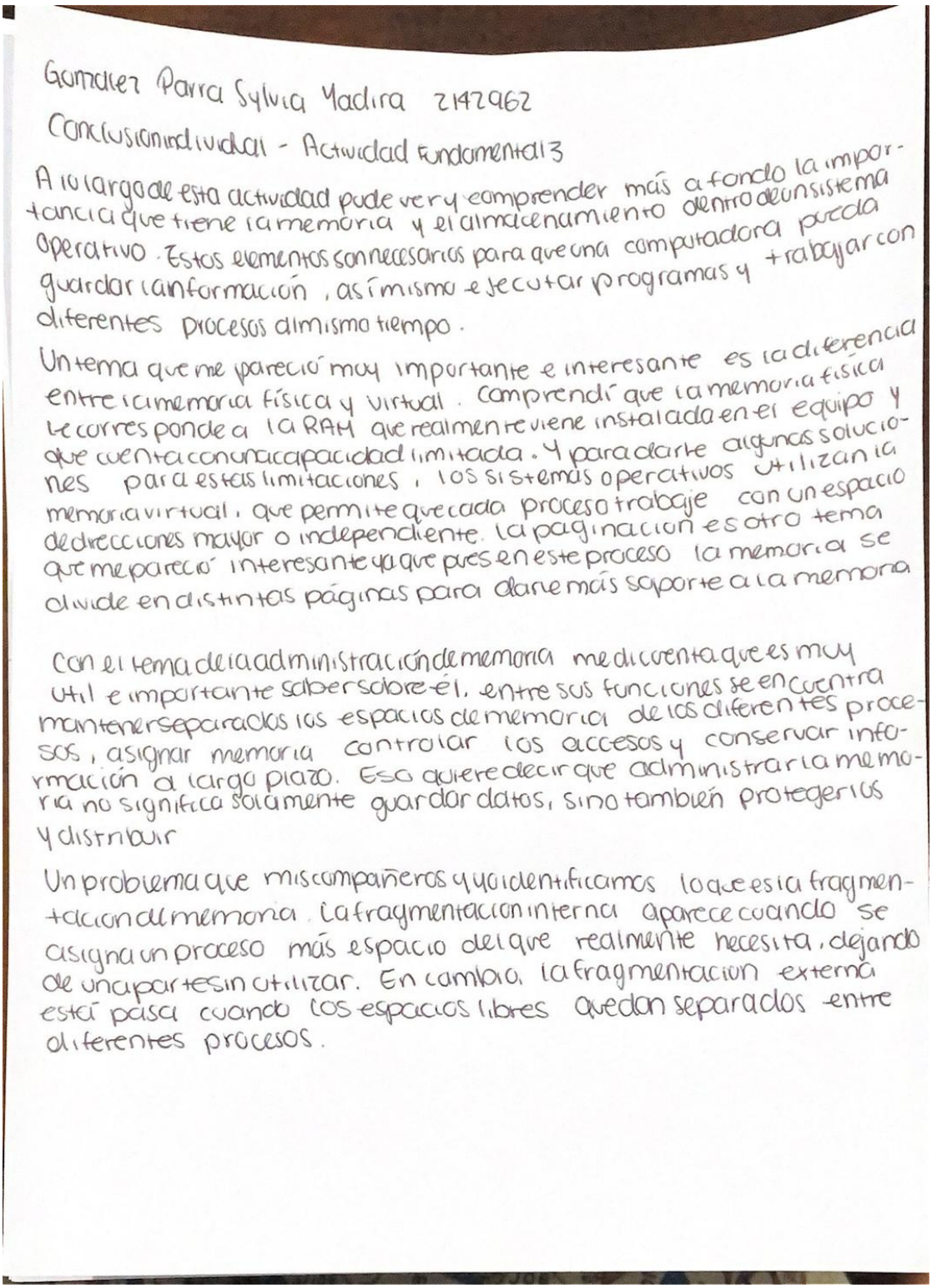
También se pudo observar que cada sistema operativo utiliza diferentes mecanismos para administrar la memoria y almacenar información, adaptándose a las características del dispositivo y a sus necesidades. Una buena administración permite mantener varios proceso funcionando correctamente, proteger la información y aprovechar mejor el espacio disponible.

Finalmente, reconocer problemas como la fragmentación interna y externa, los fallos de la página y el thrasing ayuda a comprender porque la administración de memoria es fundamental para el rendimiento y la estabilidad de un sistema. Por ello, estos conceptos son esenciales para entender como los sistemas operativos logran administrar sus recursos y ofrecer un funcionamiento eficiente al usuario.

Gonzalez Martinez Mario Ivan - 2117170

Conclusión Individual - Actividad Fundamental 3

Durante el desarrollo de esta actividad, pude comprender que la administración de la memoria y los sistemas de archivos son pilares fundamentales para el rendimiento y la estabilidad de cualquier sistema operativo. A través de las actividades que se realizaron, pude constatar que cada plataforma adapta sus algoritmos según sus necesidades de hardware: mientras los sistemas de escritorio emplean la memoria secundaria para maximizar la ejecución de procesos, las arquitecturas móviles optan por la compresión en RAM para proteger el desgaste de las memorias Flash. Comprender conceptos como la fragmentación, los fallos de página y los sistemas de archivos me otorgan una mejor visión técnica clave para mi formación en la carrera y como ingeniero.



González Parra Sylvia Yadira 2142962

Conclusión individual - Actividad Fundamental 3

A lo largo de esta actividad pude ver y comprender más a fondo la importancia que tiene la memoria y el almacenamiento dentro de un sistema operativo. Estos elementos son necesarios para que una computadora pueda guardar la información, así mismo ejecutar programas y trabajar con diferentes procesos al mismo tiempo.

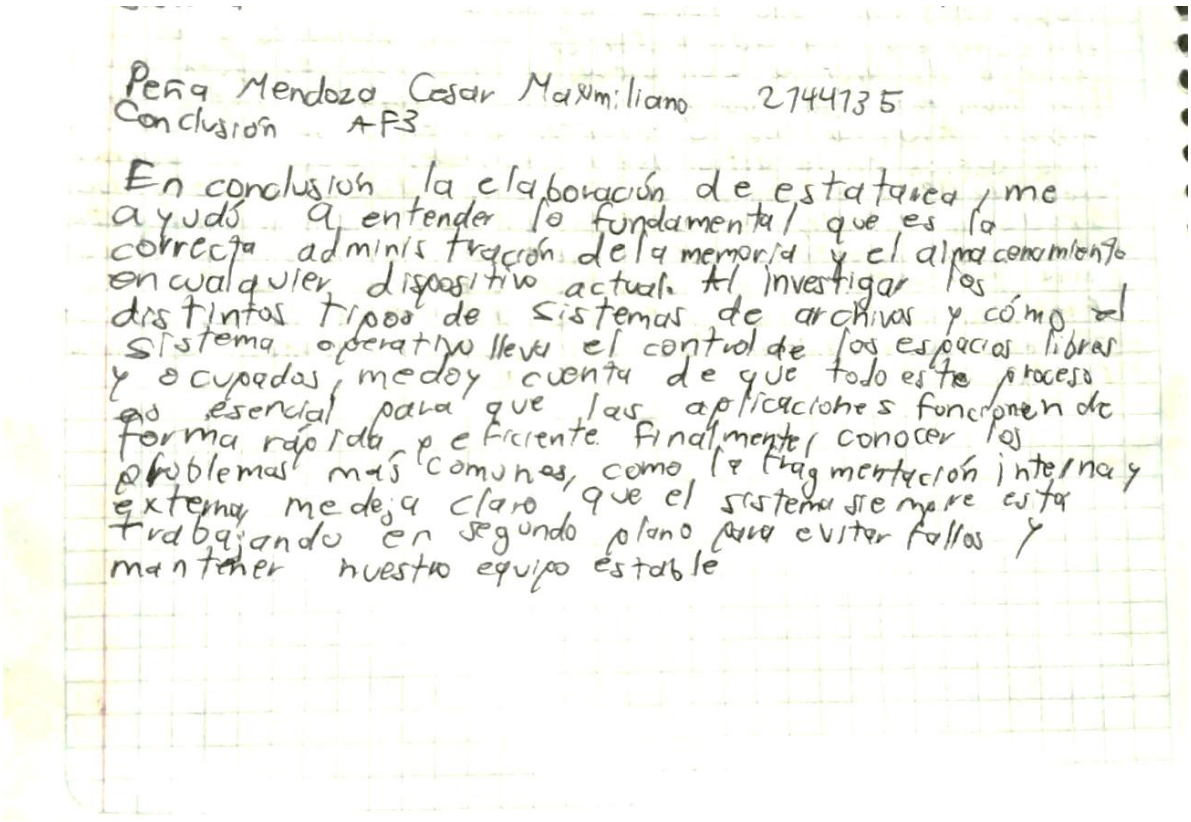
Un tema que me pareció muy importante e interesante es la diferencia entre la memoria física y virtual. Comprendí que la memoria física le corresponde a la RAM que realmente viene instalada en el equipo y que cuenta con una capacidad limitada. Y para darle algunas soluciones para estas limitaciones, los sistemas operativos utilizan la memoria virtual, que permite que cada proceso trabaje con un espacio de direcciones mayor o independiente. La paginación es otro tema que me pareció interesante ya que pues en este proceso la memoria se divide en distintas páginas para dar le más soporte a la memoria

Con el tema de la administración de memoria me di cuenta que es muy útil e importante saber sobre él, entre sus funciones se encuentra mantener separados los espacios de memoria de los diferentes procesos, asignar memoria controlar los accesos y conservar información a largo plazo. Esa quiere decir que administrar la memoria no significa solamente guardar datos, sino también protegerlos y distribuir

Un problema que mis compañeros y yo identificamos lo que es la fragmentación de memoria. La fragmentación interna aparece cuando se asigna un proceso más espacio del que realmente necesita, dejando de una parte sin utilizar. En cambio, la fragmentación externa esta pasa cuando los espacios libres quedan separados entre diferentes procesos.

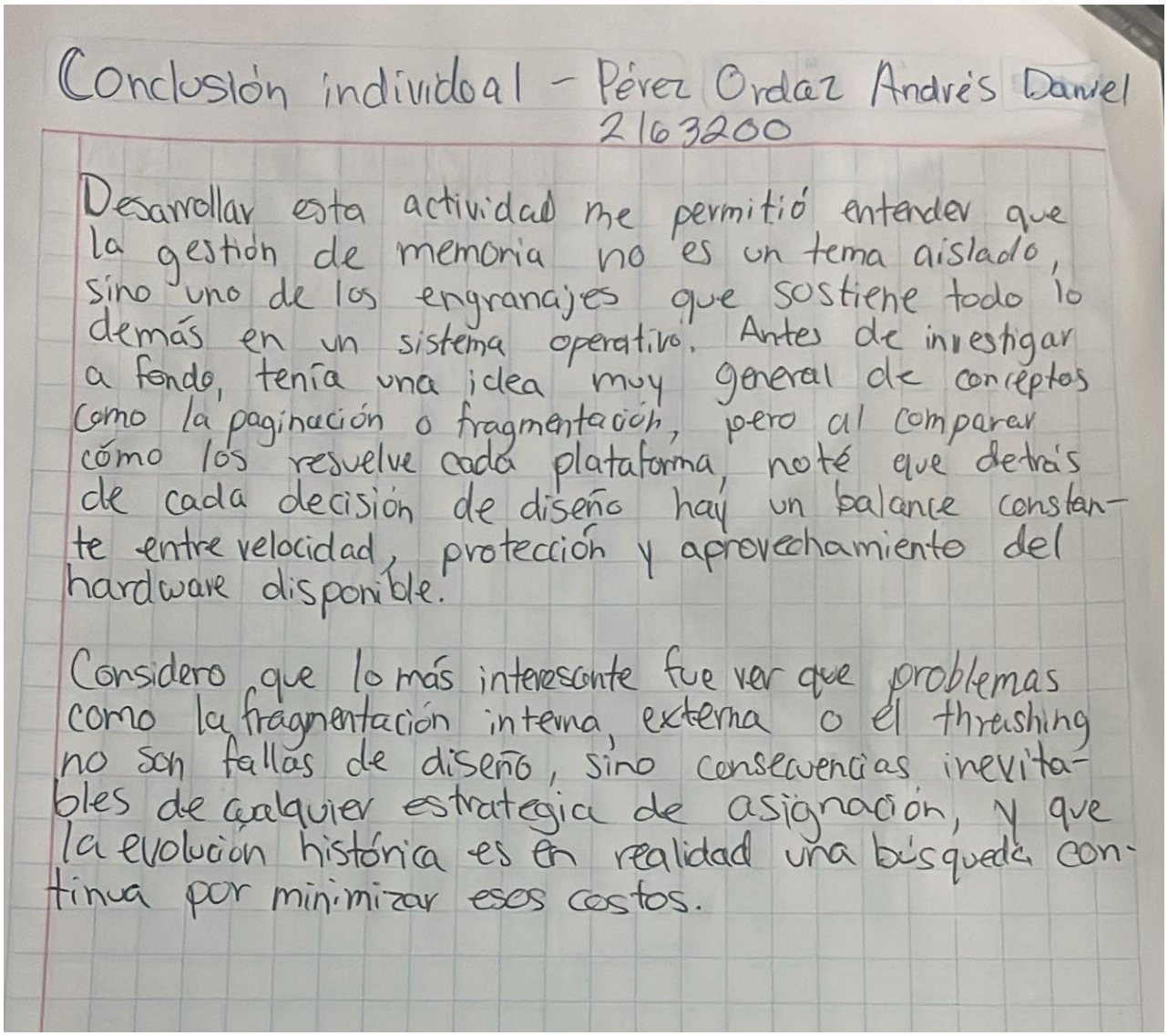
López Guzmán Eder Abihú

Esta actividad contiene conceptos muy importantes, uno de los que más me interesó fue el hecho que no exista una solución "perfecta" para la gestión de la memoria, sino que esta gestión varía en cada sistema operativo dependiendo las prioridades que este tenga y el hardware del que disponga. Por ejemplo, los sistemas de escritorio al contar generalmente con mayor capacidad de almacenamiento y procesamiento se pueden permitir algoritmos más complejos. Por otro lado, los sistemas móviles se encuentran limitados por el desgaste físico de la memoria flash, asi que optan por opciones como la compresión en RAM (zRAM) o el cierre de procesos.



Peña Mendoza Cesar Maximiliano 2144135
Conclusión AP3

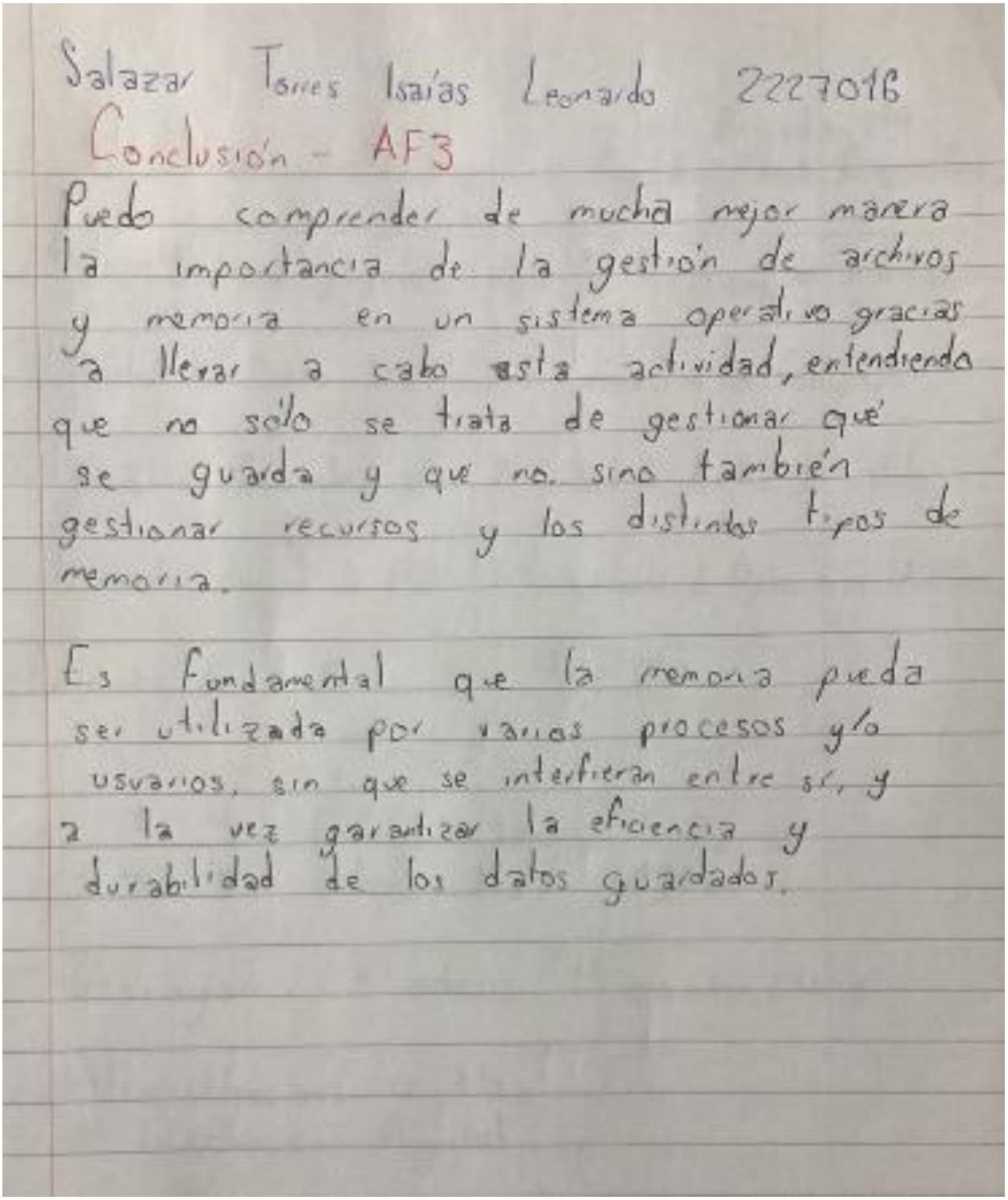
En conclusión la elaboración de esta tarea, me ayudó a entender lo fundamental que es la correcta administración de la memoria y el almacenamiento en cualquier dispositivo actual. Al investigar los distintos tipos de sistemas de archivos y cómo el sistema operativo lleva el control de los espacios libres y ocupados, me doy cuenta de que todo este proceso es esencial para que las aplicaciones funcionen de forma rápida y eficiente. Finalmente, conocer los problemas más comunes, como la fragmentación interna y externa, me deja claro que el sistema se mueve está trabajando en segundo plano para evitar fallos y mantener nuestro equipo estable



Conclusión individual - Pérez Ordaz Andrés Daniel
2163200

Desarrollar esta actividad me permitió entender que la gestión de memoria no es un tema aislado, sino uno de los engranajes que sostiene todo lo demás en un sistema operativo. Antes de investigar a fondo, tenía una idea muy general de conceptos como la paginación o fragmentación, pero al comparar cómo los resuelve cada plataforma, noté que detrás de cada decisión de diseño hay un balance constante entre velocidad, protección y aprovechamiento del hardware disponible.

Considero que lo más interesante fue ver que problemas como la fragmentación interna, externa o el thrashing no son fallas de diseño, sino consecuencias inevitables de cualquier estrategia de asignación, y que la evolución histórica es en realidad una búsqueda continua por minimizar esos costos.



Salazar Torres Isaías Leonardo 2227016

Conclusión - AF3

Puedo comprender de mucha mejor manera la importancia de la gestión de archivos y memoria en un sistema operativo gracias a llevar a cabo esta actividad, entendiendo que no sólo se trata de gestionar qué se guarda y que no, sino también gestionar recursos y los distintos tipos de memoria.

Es fundamental que la memoria pueda ser utilizada por varios procesos y/o usuarios, sin que se interfieran entre sí, y a la vez garantizar la eficiencia y durabilidad de los datos guardados.

Bibliografía

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). Fundamentos de sistemas operativos (7.ª ed.). McGraw-Hill Interamericana.

Stallings, W. (2005). Sistemas operativos: Aspectos internos y principios de diseño (5.ª ed.). Pearson Educación.

Tanenbaum, A. S. (2009). Sistemas operativos modernos (3.ª ed.). Pearson Educación de México.

Carretero Pérez, J., García Carballeira, F., de Miguel Anasagasti, P., y Pérez Costoya, F. (2001). Sistemas operativos: Una visión aplicada. McGraw-Hill/Interamericana de España.

La Red Martínez, D. L. (2004). Sistemas operativos. Universidad Nacional del Nordeste, Facultad de Ciencias Exactas y Naturales y Agrimensura, Departamento de Informática.